

/estimation-972357419.html (дата обращения: 14.04.2023).

4. Рашка С. Python и машинное обучение. — Москва: ДМК Пресс, 2017. — 418 с.

5. Гудфеллоу Я.М. Глубокое обучение. — 2-е изд. — Москва: ДМК Пресс, 2018. — 652 с.

6. Антонио Джулли, Суджит Пал Библиотека Keras — инструмент глубокого обучения. Реализация нейронных сетей с помощью библиотек Theano и TensorFlow. — Издательство Лань, 2018. — 294 с.

7. Андриевская Н.В. Статические и интеллектуальные методы прогнозирования. — Пермь: ПНИПУ, 2022. — 222 с.

8. Груздев А.В. Предварительная подготовка данных в Python. — Москва: ДМК Пресс, 2023. — 814 с.

9. Паттерсон Д., Гибсон А. Глубокое обучение с точки зрения практики. — Москва: ДМК Пресс, 2018. — 418 с.

УДК 004

ГЕНЕРАЦИЯ ТЕКСТА С ПОМОЩЬЮ НЕЙРОННЫХ СЕТЕЙ

Митина О.А.

Федеральное государственное бюджетное образовательное учреждение высшего образования «МИРЭА — Российский технологический университет», 119454, г. Москва, Проспект Вернадского, 78,

Жаров В.В.

Федеральное государственное бюджетное образовательное учреждение высшего образования «МИРЭА — Российский технологический университет», 119454, г. Москва, Проспект Вернадского, 78,

TEXT GENERATION USING NEURAL NETWORK

O.A. Mitina

Federal State Budget Educational Institution of Higher Education «MIREA — Russian Technological University» 119454, Moscow, Vernadskogo avenue, 78,

V.V. Zharov

Federal State Budget Educational Institution of Higher Education «MIREA — Russian Technological University» 119454, Moscow, Vernadskogo avenue

АННОТАЦИЯ

Проблема генерации текста является актуальной, поскольку языковые модели нейронных сетей входят в нашу жизнь очень активно. Это и поисковики, отвечающие на запросы пользователей, и автоматическое перефразирование текста в помощь копирайтерам. Этим занимается область обработки естественного языка, NLP, куда входит генерация текста на основе запроса пользователя. Именно генерация текста с помощью нейронных сетей имеет преимущество перед другими способами генерации. В ней заложена самостоятельная реакция на нестандартные запросы. Это перспективное направление является самым малоизученным во всей линейке инструментов генерации текста.

Чтобы улучшить качество итоговой генерации, разработчик, помимо обычного обучения нейронной сети, может использовать также и «пост-обучение». В качестве примера можно привести генеративно-состязательную сеть, в которой генератором выступает обученная ранее модель.

ABSTRACT

The problem of text generation is relevant, because language models of neural networks are very active in our lives. These are search engines that respond to user requests, and automatic text paraphrasing to help copywriters. This is the area of natural language processing, NLP, which includes generating text based on a user request. Text generation using neural networks has an advantage over other generation methods. It contains an independent response to non-standard requests. This promising direction is the least studied in the entire line of text generation tools.

To improve the quality of the final generation, the developer, in addition to the usual training of the neural network, can also use "post-training". An example is a generative adversarial network in which a previously trained model acts as a generator.

Ключевые слова: задача генерации текста; нейронные сети-трансформер, генеративно-состязательная сеть.

Keywords: generation text problem; transformer neural networks, GAN.

Введение

Объектом нашего исследования выступает NLP (Natural Language Processing), являющаяся

областью разработкой искусственного интеллекта. NLP обрабатывает человеческий язык, то есть естественный язык, развивавшийся

на протяжении тысячелетий как основа коммуникации между людьми. Человеческие языки ограничены рядом правил, зависящих от эпохи и места проживания сообщества людей. Отличие естественного языка от формального (формулы, язык программирования и т.д.) заключается в том, что ошибки не сильно влияют на понимание. Поэтому строго детерминированный процесс анализа текста машинными алгоритмами отсутствует.

Сегодня известно три способа генерации текста. Первый: поисковый способ, когда задача заключается не в генерации, собственно, текста, а в выборе соответствующего из заранее подготовленной линейки. Второй: шаблонный способ. При нём используется трафарет, куда подставляются слова с нужными характеристиками (согласование по падежу, роду, числу и т.п.). Недостатком способа является требование создать много трафаретов в зависимости от фразы, поданной на вход. Третий способ: генерация текста с помощью нейронных сетей.

В данной работе мы рассматриваем третий способ (генерация с помощью нейронных сетей), поскольку он является самым современным. Для обучения нейронных сетей не нужна предварительная разметка текста. Также третий способ характеризует хорошее качество генерируемых ответов. Генерация с помощью нейронных сетей позволяет учитывать встречаемость слов и отношение их смыслов. Также мы можем отметить, что, обучив одну нейросеть на комплексном тексте, мы одновременно получаем решение для нескольких прикладных задач.

Однако у третьего способа имеются и недостатки. Обучение хороших языковых моделей требует невероятных мощностей и большого количества машинного времени. Всё это доступно только крупным компаниям. Также к недостаткам можно отнести требования больших массивов текста для обучения модели.

Поскольку нейросеть не может воспринимать напрямую символы разработчику нужно заранее обработать текст перед тем, как подавать его на вход. Для этого используется токенизация, то есть

деление исходного массива текста на семантические единицы.

Самой простой техникой извлечения признаков для работы с текстом является мешок n-грамм. Данный метод описывает вхождение n-граммы в текст. В начале определяется размер словаря, куда будут добавляться наиболее часто встречающиеся семантические единицы.

Преимущества мешка n-грамм заключается в следующем. Первое: токенизация по n-граммам выделяет часто повторяющиеся в конкретном тексте морфемы и имена собственные, что понижает количество ошибок за счёт статичных токенов.

Второе: словарь задаётся для конкретного текста при предобработке. Размер словаря можно регулировать в качестве гиперпараметра.

Третье: токен-морфема позволяет увеличить количество примеров для каждого токена. Например, окончание глаголов «ться» встречается чаще, чем глагол с таким окончанием. При этом рецептивное поле расширяется не так сильно, как при токенизации по одним лишь символам.

Четвёртое: токенизация по n-граммам устойчива к ошибкам и опечаткам, потому что ошибка не добавляется автоматически в словарь, как при простой токенизации.

Данный метод уменьшает количество ошибок при построении частотных для языка слов, поскольку в нём уделяется большое внимание грамматике и сохраняется смысл фраз.

Цель исследования

Реализация модели генерации текста, продолжающая по смыслу фразу на основе корпуса текстов с помощью языка программирования Python.

Материал и методы исследования

В наши дни большинство языковых моделей основывается на архитектуре трансформера. Это считается самым эффективным способом обработки запросов на естественном языке, что стало возможным благодаря технологии self-attention. Данная технология выделяет во фразе ключевые понятия и смысловые фразы.

Рассмотрим работу механизма внимания более наглядно (Рисунок 1).

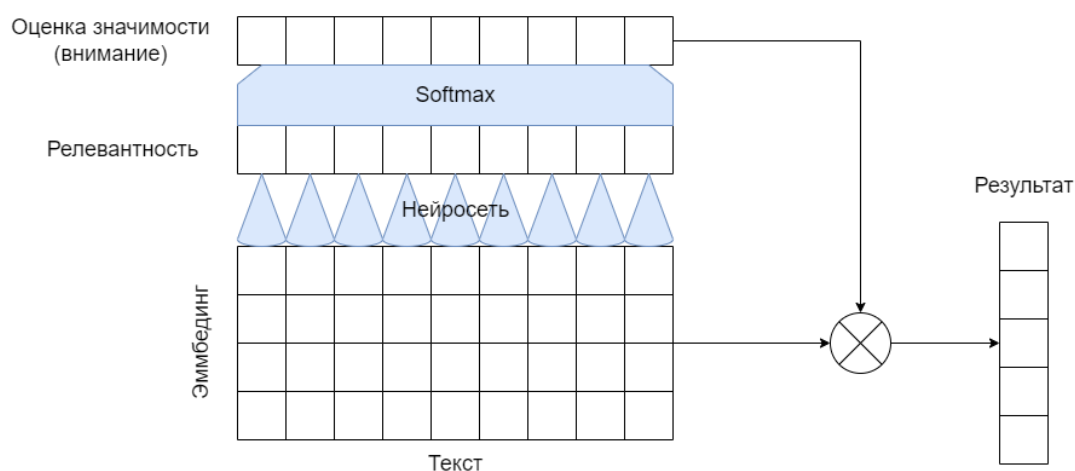


Рисунок 1 — Схематическое изображение внимания

Матрица, чья размерность равна длине текста на размер эмбединга, подаётся на вход механизма внимания. Мы применяем небольшую нейросеть к вектору каждого токена. У нейронной сети один выход — это число, указывающее на то, насколько значимым является этот токен. Оно тем больше, чем выше эта значимость. Данная релевантность наиболее часто рассчитывается с помощью однослойной нейронной сети. Необходимо рассчитывать релевантность с помощью одной и той же модели, поскольку её веса не зависят от конкретного слова и его позиции в тексте.

После того как оценена релевантность для каждого слова, оценки нормируются с помощью функции softmax. После этого оценки значимости умножаются на соответствующие слова входных

векторов. Получается результирующий вектор, чья размерность равняется размеру эмбединга слов.

Последовательность, которая подаётся на вход, состоит слов, релевантность которых зависит от запроса пользователя. Например, при сравнении двух текстов, вектор-запрос отвечает за то, что ищет пользователь, а матрица — за то, где ищут ответ. Вектор релевантности является перемножением двух матриц для поиска их схожести.

Однако запрос и ответ как два отдельных текста есть не всегда, но их можно получить с помощью двух нейронных сетей с разными весами (Рисунок 2). Этот процесс называется механизмом внутреннего внимания (self-attention).

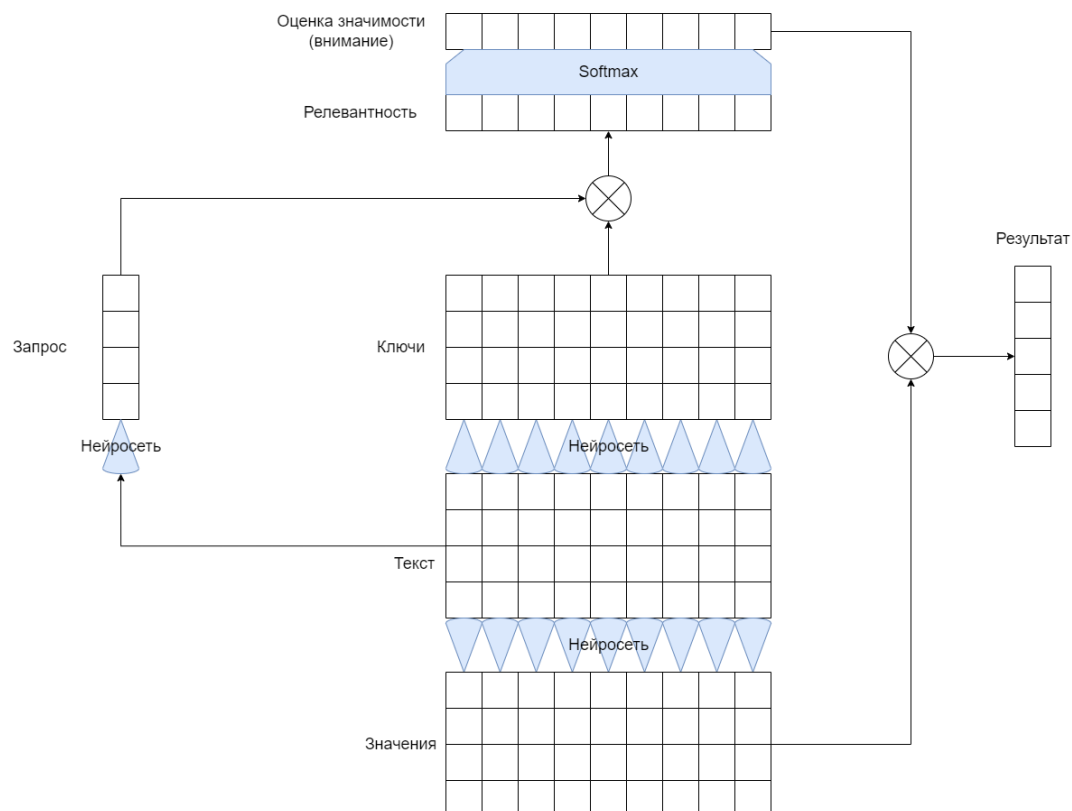


Рисунок 2 — Схематическое изображение внутреннего внимания (self-attention)

В 2017 году была изобретена модель, получившая название «трансформер». [1.1] На ней базируются все современные большие языковые модели.

На Рисунке 3 представлена внутренняя архитектура трансформера, основанного на механизме внутреннего внимания.

Трансформер принимает матрицу признаков токенов (текст). Благодаря применению внимания получается матрица такого же размера, что и

входная матрица, но при этом каждый её вектор содержит информацию о значении соответствующего токена в контексте всех остальных токенов.

Особенность алгоритма работы внутреннего внимания заключается в том, что одни и те же данные используются в качестве запросов и в качестве ключей. Данными являются токены из изначального текста, поданного на обработку.

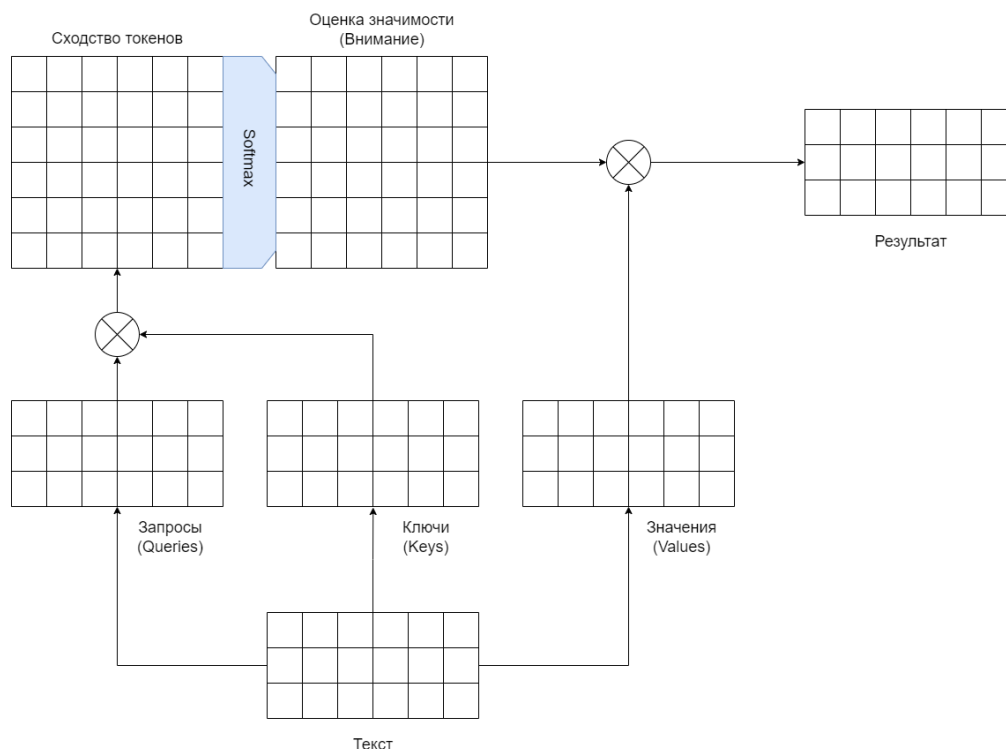


Рисунок 3 — Схема работы части трансформера

Тем самым, на первом шаге происходит преобразование входной матрицы токенов с помощью трёх независимых нейросетей с разными весами. В итоге мы получаем три матрицы: запросы (queries), ключи (keys) и значения (values), как показано в формуле (1).

$$\begin{cases} Q = w_Q * T \\ K = w_K * T, \\ V = w_V * T \end{cases} \quad (1)$$

где Q — матрица запросов Queries;
 w_Q — веса нейросети для создания матрицы Q ;
 T — входная матрица, текст;
 K — матрица ключей Keys;
 w_K — веса нейросети для создания матрицы K ;
 V — матрица значений Values;
 w_V — веса нейросети для создания матрицы V .

Перемножим матрицу запроса и ключи, мы получаем матрицу попарного сходства каждого токена с каждым. Следующим шагом будет

нормирование матрицы по столбцам (по строкам) с помощью функции softmax. Далее эта матрица перемножается с матрицей значений. Итоговый результат преобразуется с помощью небольшого полносвязного слоя, как показано в формуле (2).

$$\text{Logits} = Q * K^T;$$

$$S = \text{softmax}(\text{Logits});$$

$$\text{Result} = w_R * S;$$

$$\text{Output} = w_O * \text{Result}, \quad (2)$$

где Logits — матрица сходства токенов, Result — матрица результата работа блока, w_R — веса нейросети для создания Result, S — матрица оценки значимости (внимания), Output — выходная матрица, w_O — веса нейросети для создания Output.

Механизм внутреннего внимания обладает как достоинствами, так и недостатками.

Последние заключаются в том, что операция усреднения остаётся грубой, когда в процессе теряется много информации.

Например, если один токен получается большим вес, то все остальные автоматически получают меньший вес, даже если содержат полезную информацию. Подобный механизм внимания учитывает лишь один аспект и сравнивает токены один раз, хотя они могут различаться множеством характеристик.

Механизм внутреннего внимания не рекомендуется использовать как единственное выразительное средство в нейросети, поскольку он теряет информацию.

Другим недостатком механизма внимания является в том, что он ничего не знает о занимаемой токеном позиции относительно друг друга. Для строго детерминированных языков (например, для английского языка) очень важен порядок слов в предложении. Но и для других языков позиция слов во фразе также несёт смысловую информацию.

Для того чтобы нивелировать первые две проблемы, предлагается модифицировать механизм внимания, то есть ввести несколько «голов». [1.2] Каждая из голов будет обучаться независимо и получит уникальные веса и сможет чуть иначе проанализировать текст.

Для устранения третьей проблемы используется позиционное кодирование. [3]

Поскольку код позиции имеет ту же размерность, что и эмбединг, их можно сложить по формуле (3).

$$\begin{aligned} T(t_i) \\ = \text{Emb}(t_i) + \text{PosCode}(i), \text{PosCode}(i) \\ \in \mathbb{R}^{\text{EmbSize}}. \end{aligned} \quad (3)$$

Периодические функции, вроде синусов и косинусов с разными частотами, используются для

автоматической генерации кода позиции, что допустимо для любой длины эмбединга и входной длины текста.

Механизм внимания решает очень важную задачу: при генерации текста он даёт возможность управления зависимостью. Подавая для обучения текст и желая, чтобы модель сгенерировала токен для, например, четвёртой позиции, нужно заставить её не учитывать последующие позиции (четвёртую, пятую и т.д.), а только первую, вторую и третью. Для этого рекомендуется создавать маску.

Маска должна быть той же размерности, что и матрица сходства токенов, а именно: квадратная со стороной, равной длине входной последовательности. Если сходство токенов нужно учитывать, мы ставим единицу на соответствующей позиции маски, и ставим ноль, если сходство не учитывается. Маску применяют до использования функции softmax, что позволяет превращать необходимые сходства токенов в минус бесконечность, из-за чего они становятся равными нулю и не учитываются после применения нормализации.

Ключевая идея трансформера — это внутреннее внимание с несколькими головами. Однако трансформер состоит из нескольких блоков. Первый блок — внимание, используемое для учёта глобального контекста. Второй блок — преобразованные с помощью двухслойной нейронной сети признаки каждого токена. Третий блок — это связи в обход нелинейности, ускоряющие процесс обучения за счёт лучшего протекания градиентов. Эти блоки ставятся один поверх другого. Схема такого слоя представлена на Рисунке 4.

Архитектура модели, генерирующая текст, базируется на архитектуре трансформера и, конкретно, на энкодере, одной из его частей. Модель состоит из 16 «голов внимания» и 5 слоёв.

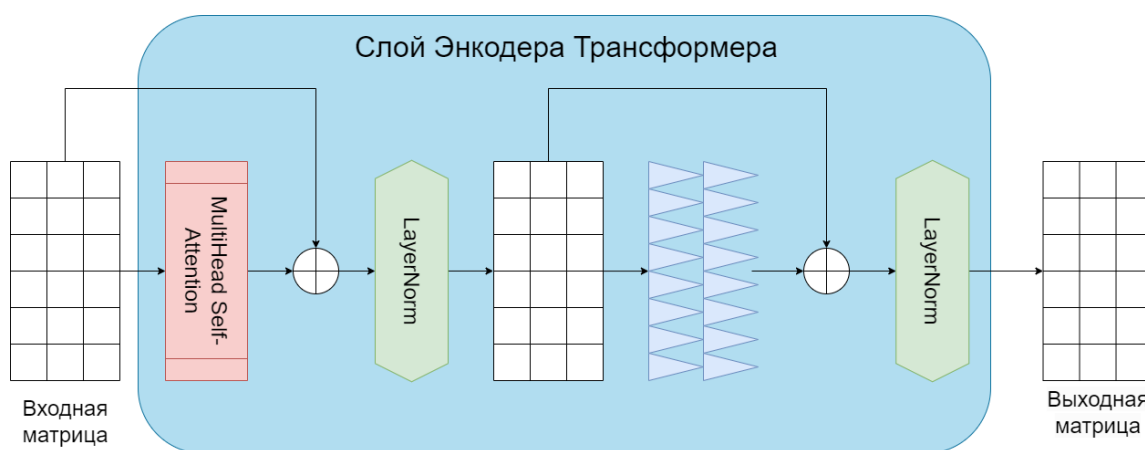


Рисунок 4 — Схема энкодера трансформера

Для того, чтобы текст выглядел более натурально, ранее обученную модель мы используем её в качестве генератора в генеративно-сопоставительной сети (Generative Adversarial Networks, GAN).

Теперь рассмотрим, как работает генеративно-сопоставительная сеть. Она состоит из двух сетей: сеть-генератор, которая генерирует предложение, и сеть-дискриминатор, которая определяет, откуда взято предложение, сгенерировано ли оно или взято из обучающего набора.

Сеть-генератор учится обманывать сеть-дискриминатор, за счёт чего создаёт более реалистичные предложения. В свою очередь сеть-дискриминатор адаптируется в реальном времени к увеличивающейся способности сети-генератора создавать реалистичные предложения. [Error! Reference source not found.]

Архитектура нейросети-дискриминатора состоит из следующих блоков:

- слой эмбединга;
- рекуррентная сеть LSTM с внутренним размером равным 100;
- полносвязный слой с количеством нейронов на выходе равным одному.

Выход последнего линейного слоя равняется одному нейрону поскольку существует лишь одна переменная «тумблер»: предложение фальшивое (0) или настоящее (1).

Из этого следует предварительные этапы обучения данной архитектуры:

- создание словаря токенов на основе исходного корпуса текстов;
- деление текста в соотношении 2 к 1: первая часть для обучения «главной» сети на основе трансформера, вторая часть для обучения генеративно-состязательной сети. Так же делим первую выборку на обучающую и тестовую;
- обучение нейронной сети на основе трансформера;
- обучение генеративно-состязательной сети с генератором в виде нейросети на основе трансформера и с дискриминатором в виде сети на основе LSTM;
- оценка итоговой генерации и метрики её метрика качества.

Для обучения нейронной сети на основе трансформера мы предлагаем использовать функцию потерь «перекрёстная энтропия» (Cross-Entropy Loss). Она считается по следующей формуле (4):

$$CE_Loss = -\frac{1}{n} \sum_{i=1}^n (y_i * \ln(\hat{y}_i) + (1 - y_i) * \ln(1 - \hat{y}_i)), \quad (4)$$

где y_i — истинное значение, $\hat{y}_i$ — предсказанное значение, n — количество классов.

Для обучения генеративно-состязательной сети мы предлагаем использовать функцию «бинарная перекрёстная энтропия» (Binary Cross-Entropy Loss). Её расчёт представлен в формуле (5).

$$BCE_Loss = -y * \ln(\hat{y}) + (1 - y_i) * \ln(1 - \hat{y}), \quad (5)$$

где y — целевое, «истинное» значение, $\hat{y}$ — предсказанное значение.

Сгенерированный текст предлагается оценивать с помощью MAUVE, которая вычисляет информационные расхождения двух текстов в пространстве эмбедингов. [1.5]

Для построения оценки используется KL-дивергенция (расстояние Кульбака-Лейблера). Вычисление кривой, площадь под которой и будет являться MAUVE-оценкой, представлено в формуле (6). Оценка MAUVE — это дробное число от 0 до 1.

$$R_\lambda = \lambda P + (1 - \lambda)Q, \lambda \in (0, 1),$$

$$KL_P = KL(P|R_\lambda) = \sum_x P(x) \log \frac{P(x)}{R_\lambda(x)},$$

$$C = \left\{ \left(\exp(-c * KL_Q), \exp(-c * KL_P) \right) \right\} \quad (6)$$

где c — гиперпараметр для масштабирования, Q — распределение текста, сгенерированного моделью, P — распределение текста, написанного человеком.

В данной работе в качестве данных выбрана трилогия «The Lord of the Rings» Джона Рональда Руэла Толкина. [1.6]

Чтобы создать собственный словарь токенов на основе подготовленного текста, нами была использована библиотека для языка Python

youtokentome. Размер словаря — тысяча токенов. Часть построенного словаря отображена на Рисунке 5. Символом "_" отображён пробел для лучшего восприятия.

В словаре много целых слов, начинающихся с пробела («_was», «_dark», «_great»). Имеются и целые имена («_Sam», «_Boromir», «_Gandalf»), также начинающиеся с пробела. Пробел перед словом означает, что такой токен не может оказаться посередине слова.

<PAD> <UNK> <BOS> <EOS> _ e t a o n h s r i d l u w g m f y c , . b p ' k v I T B S A ! F
H G W ; - M ` E ? : _ R N O L D Y P q C j x z " - U) (K V 1 Q J 9 4 3 2 0 7 6 8 5 Z * X =
/ _ t h e _ a _ t h e i n _ s _ w n d r e _ o _ h o u _ b _ f _ a n d e r e d i n g a t l l i t _ m _ c _ l _ o f i s
a r o r e n _ d a s _ t o _ t h _ n e s r o _ h e _ g a n i d _ i n o n o w _ ' _ I a d u t _ p _ b e o t _ y _ T g h
o m l d _ e _ i t _ s t _ s a _ w a s _ w e _ t h a t _ u v e _ y o u l e i m i r a y _ o n _ T h e l y g h t _ h a a m e t _ B
s t _ n o t _ S _ s a i d _ h i s i l l o o s e _ a s _ t h e y v e r _ r _ u r _ f o r _ A _ F a c i t h _ i s e e _ G a n d _ w h
_ s h a l i l i g h t _ H _ w i t h a i n a l l a r d _ r e , ' k e o u l d . ' _ h a d _ M _ a n e n t h e r _ a t i c o r e s ,
i f o m e _ ` a s t _ h a v e _ w e r e _ h i m s . r e d r o d _ b u t _ F _ u p _ W a g i n d _ t h e m o n g _ k t h r o d o _ l e
_ F r o d o r o m _ o r _ d o _ a l l e . e , _ s e e _ H e _ f r o m n t p p _ f e o o d c h _ n e a c k _ a r e r i _ t h e i r t e r
' s _ g r e _ s p _ l i _ R _ h o _ t h e r e _ n o _ g o _ n o w _ s u _ a g _ m e _ o u t o u n d _ w i l l _ B u t o r n _ s o
_ i f o o k ! ' e s s u m o r d r y o w n a m e c e _ k n _ a l _ O _ w h e o u g h o u t _ l o n g t o u s t o l _ I _ l e l l
a r k _ m y _ r o v e n _ c l _ a b _ a g a i n _ v _ b y a k a l f a n g h e d _ s e o u r w a y i d e t h e r a n y e s t v e s e d .
a n d a l f _ d o w n _ l o o k b b _ D _ c h _ G a n d a l f _ y o u r _ t h i s n d e r _ c o u l d a p _ i n t o _ l i k e u n _ c a n
w a r d _ d e ' t _ N _ I t _ w o u l d r a e d , u r n r o w _ c a m e q u e r e d i r e _ m o r e b b i t l f _ T h e y e r s o n e
_ s o m e i o n _ S a m l i ? ' _ s e e m r e e d e n _ u s f o r e _ b e e n _ P e s , _ b a c k e e p _ w h a t a s s o l l a r t c k
_ o v e r _ a m a u _ a n y o u g h t i s t a n t _ g r e a t _ d a r k _ a w a y _ f l _ c o m e i n k h a t _ c o m _ b l _ A n d _ T

Рисунок 5 — Часть словаря токенов для текста *The Lord of the Rings*

С помощью данного словаря можно токенизировать всю выборку текстов. Однако после токенизации надо убедиться, что токен <UNK> в выборке не встречается и все символы в тексте представлены в словаре.

После токенизации длина предложений может меняться, они становятся неравными. Поэтому

нужно найти новую, оптимальную длину. Строится гистограмма распределения длин фрагментов, где по оси x откладывается длина предложения, а по оси y — количество фрагментов такой длины (см. Рисунок 6).



Рисунок 6 — Гистограмма распределения длин фрагментов в токенах

Длина, равная 100, является оптимальной, поскольку это центр «колокола» получившегося распределения. В таком варианте соблюдается «баланс» между количеством предложений с паддингами в конце до нужной длины и предложений с укороченной его частью.

Следующим этапом, идущим за обучением сети, является генерация текста с помощью лучевого поиска. Он рассматривает несколько вариантов последующих токенов. На каждом уровне лучевой поиск генерирует всех преемников состояний на текущем уровне, сортируя их в порядке возрастания эвристической стоимости. Алгоритм использует ограниченное количество лучших состояний на каждом шаге. Этот гиперпараметр называется «шириной луча».

Лучевой поиск в качестве единственного метода может давать предсказуемые и повторяющиеся ответы, особенно если модель

«уверена в себе» (англ. overconfident). В этом случае выбор следующего токена по разнице весов очевиден и распределение вероятности выбора происходит с минимальной энтропией. Чрезмерная «уверенность» модели приводит к закликиванию, что снижает качество генерированного текста.

Для управления энтропией выбора следующего токена в генерации используется «перевзвешивание» изначального распределения весов с помощью температуры и сглаживания меток. [1.7]

Способ изменения весов со сглаживанием и температурой представлен в формуле (7).

$$W = \frac{\ln \left((1 - \alpha) * w + \frac{\alpha}{\text{len}} \right)}{t}, \quad (7)$$

где w — изначальное распределение весов, t — параметр температуры, α — параметр сглаживания, len — количество токенов.

Чем больше параметры температуры и сглаживания (которые лежат в диапазоне от 0 до 1), тем больше энтропия и более непредсказуем генерируемый текст.

При оценке результатов рекомендуется исследовать разные величины случайности при генерации. Температура и сглаживание являются гиперпараметрами, их оптимальную величину для каждой конкретной модели следует определять путём перебора.

Приведём пример итоговой генерации текста после полного цикла обучения нейронной сети. Было задано четыре начала фразы: «Today we are», «For the King and», «We are going» и «That long story is».

Предложения, которые начинаются с фразы «Today we are», модель в основном продолжает глаголами:

- «Today we are going to the Ford of Isen»;
- «Today we are attack Ugluk»;
- «Today we are fighting on the hill-top».

Такие варианты не противоречат заложенному смыслу.

Продолжение фразы «For the King and» модель генерирует как перечисление имён собственных:

- «For the King and the Dwarf, and the Dark Lord of the Mark Lordor»;
- «For the King and Gladorfindel, and the Dark Lord of Mordor»;
- «For the King and the Lady of Rohan, and the Galadhrim of Gondor, and the Third Marshal».

Подобное перечисление выглядит громоздко, словно модель не знает, когда надо «остановиться».

Начало фразы «We are going» модель чаще всего продолжает как «to Mordor». Примечательно, в книге «Властелин Колец», на основе которого обучалась модель, это позиционируется как главная цель — дойти до Мордора.

Вариант «That long story is» продолжается как:

- «That long story is according to the Bridge of Mordor.»;
- «That long story is told of the Enemy of the Third Mountains»;
- «That long story is told of the Battle of Minas Tirith»;
- «That long story is the oldest of the Westfold of the Elder Days».

Нужно отметить, что это характерная для начала какой-либо истории фраза продолжается обозначением места, времени, события или персонажа, о которых может идти повествование.

При низких значениях температуры наборы токенов могут повторяться. Например: «We are going tomorrow.' 'Yes, yes,' said Gollum. 'No, no, no, no!' said Gollum. 'No, no!' said Gollum. 'No».

При высоких значениях температуры случайность токенов повышается, из-за чего предложение может выглядеть неестественным. Например: «That long story is known to us; but it is

not far from the Eryn Muir.' 'Yes,' said Gimli. 'Maybe not,' said Legolas; 'but I».

Данное наблюдение применимо и к параметру сглаживания, когда при значении от 0.5 и выше генерируется плохо связанный друг с другом набор токенов. Например: «Today we are lost.' The hobbits looked at themselves and smiled. There was no answer. They were only a few prisoners, and they had not seen themselves in the».

Выводы

Итак, оптимальные гиперпараметры температуры и сглаживания подбираются в соответствии каждому предложению. К примеру, модель генерирует продолжения фразы «For the King and» одинаковыми по структуре, и лучшая генерация происходит при высоких значениях обоих гиперпараметров, от 0.4 и выше. Тогда как для фразы, начинающейся с «That long story is» оптимальные тексты генерируются при средних значениях, а именно 0.3 для температуры и 0.1 для сглаживания.

Средняя оценка MAUVE для модели до обучения на основе генеративно-состязательной сети составляет 0.317, а после — 0.332. После генеративно-состязательной сети отмечается небольшое улучшение качества генерации модели, которое может быть объяснено малым количеством данных, на которых обучалась эта сеть. Чтобы не переобучиться на существующих данных, дискриминатор каждый раз должен получать разные продолжения фраз.

Для иллюстрации данной работы использовалась модель с ограниченным объёмом текстов. Чтобы улучшить результаты, следует увеличить модель. Текущая модель обладает лишь 3.15 млн. параметров, тогда как современные большие языковые модели используют сотни миллиардов. Увеличение количества параметров модели, её слоёв позволяет находить большее число зависимостей в тексте, а также обрабатывать более длинные последовательности. Однако следует учитывать, что на обучение большой модели уйдёт много времени (до нескольких месяцев) даже при условии, что процесс будет идти на хороших мощностях и профессиональном оборудовании.

Список литературы

1. Attention Is All You Need / Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. — Режим доступа: <https://arxiv.org/pdf/1706.03762v4.pdf>
2. Attention and its Different Forms / Anusha Lihala. — Режим доступа: <https://towardsdatascience.com/attention-and-its-different-forms-7fc3674d14dc>
3. Transformer Architecture: The Positional Encoding / Amirhossein Kazemnejad. — Режим доступа: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

4. Франсуа Шолле. Глубокое обучение на Python. — СПб.: Питер, 2018. — 400 с.
5. Mauve: Measuring the Gap Between Neural Text and Human Text using Divergence Frontiers / Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, Zaid Harchaoui. — Режим доступа: <https://arxiv.org/pdf/2102.01454.pdf>
6. Lord of the rings text / Ashishsinha. — Режим доступа: <https://www.kaggle.com/datasets/ashishsinhai/lord-of-the-rings-text>
7. What is Label Smoothing? / Wanshun Wong. — Режим доступа: <https://towardsdatascience.com/what-is-label-smoothing-108debd7ef06>